

Reviewer Pack — Minimal Matroid Rank of Affine Geometric Loci vs Monotone Ci...

Entry 536c41f3714d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 07:18:20 UTC

Minimal Matroid Rank of Affine Geometric Loci vs Monotone Circuit Lower Bounds for k-CLIQUE

Entry ID: 536c41f3714d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 07:18:20 UTC

1. Conjecture

Field A (mathematical branch): Affine Geometry **Field B** (complexity object): Monotone Circuit Complexity for k-CLIQUE

Statement:

[‘The minimal rank of the matroid representing the affine geometric loci of a k-CLIQUE instance is at least as large as the size of the smallest monotone circuit computing k-CLIQUE, i.e., $E[\text{rank}(M)] = \Omega(n^k \log n)$, where M is the matroid and n is the number of vertices.’, ‘For all instances with k vertices, the minimal rank of the matroid representing their affine geometric loci is lower bounded by the smallest monotone circuit size for k-CLIQUE, i.e., $\min_{\{M\}} \text{rank}(M) \geq n^k \log n$.’]

Rationale (proposer’s reasoning):

[‘Affine geometry provides a rich framework to study geometric objects and their representations. Matroid theory offers a combinatorial abstraction of these geometric structures. The conjecture leverages the connection between matroids and monotone circuits, suggesting that the complexity of k-CLIQUE can be characterized by affine geometric properties.’, ‘This bridge may expose new structural insights into the hardness of k-CLIQUE, potentially leading to novel circuit lower bounds or proving the conjecture itself by connecting the geometric nature of the problem with its computational difficulty.’]

Taxonomy category: MONOTONE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d6b6775b654ba53f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

If the mean minimal matroid rank across all k-CLIQUE instances, computed from 30 independent random seeds, is at least $n^k \log n$ for each instance, then the conjecture is supported. If any seed produces a mean minimal matroid rank below $n^k \log n$, or if any

seed produces an individual minimal matroid rank greater than $n^k \log n$, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "affine geometry" AND "monotone circuit complexity" AND "k-CLIQUE" AND "matroid rank" - "matroid representing affine geometric loci" AND "lower bound" AND "monotone circuit size" - " $E[\text{rank}(M)] = \Omega(n^k \log n)$ " AND "k-CLIQUE"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_clique(n: int, k: int):
        if n < k:
            return []
        vertices = list(range(n))
        clique_edges = [(u, v) for u in range(k) for v in range(u + 1, k)]
        remaining_edges = [(u, v) for u in range(k, n) for v in range(u + 1, n) if (u, v) not in clique_edges]
        random.shuffle(remaining_edges)
        edges = clique_edges + remaining_edges[:n - k]
        return vertices, edges

    def matroid_rank(edges):
        n = len(edges)
        if n == 0:
            return 0
        rank = 1
        for i in range(n):
            new_edge = edges[i]
            if all((new_edge[0] != edge[0] and new_edge[0] != edge[1]) or (new_edge[1] != edge[0] and new_edge[1] != edge[1]) for edge in edges[:i]):
                rank += 1
        return rank
```

```

def monotone_circuit_size(k):
    # Upper bound on the size of a monotone circuit computing k-CLIQUE
    return math.factorial(k) * math.log2(math.factorial(k))

n = random.randint(5, 40)
k = random.randint(3, min(n - 1, 8))
vertices, edges = generate_k_clique(n, k)
rank = matroid_rank(edges)
circuit_size = monotone_circuit_size(k)

return {
    "metric_name": "matroid_rank",
    "metric_value": rank,
    "instances_tested": 1,
    "conjecture_holds": rank >= circuit_size,
    "counterexample": "" if rank >= circuit_size else f"rank={rank} < {circuit_size}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_rank = sum(result["metric_value"] for result in results) / len(results)
    std_dev = math.sqrt(sum((result["metric_value"] - mean_rank) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_dev} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"rank < monotone circuit size\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

88.00841267189'}
TRIAL: {'metric_name': 'matroid_rank', 'metric_value': 31, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'matroid_rank', 'metric_value': 28, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'matroid_rank', 'metric_value': 18, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'matroid_rank', 'metric_value': 8, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'matroid_rank', 'metric_value': 31, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'matroid_rank', 'metric_value': 29, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'matroid_rank', 'metric_value': 29, 'instances_tested': 1, 'conjecture_holds': True}
TRIAL: {'metric_name': 'matroid_rank', 'metric_value': 41, 'instances_tested': 1, 'conjecture_holds': True}

```

```

TRIAL: {'metric_name': 'matroid_rank', 'metric_value': 26, 'instances_tested': 1, 'conjecture_holds': 1}
TRIAL: {'metric_name': 'matroid_rank', 'metric_value': 42, 'instances_tested': 1, 'conjecture_holds': 1}
TRIAL: {'metric_name': 'matroid_rank', 'metric_value': 29, 'instances_tested': 1, 'conjecture_holds': 1}
TRIAL: {'metric_name': 'matroid_rank', 'metric_value': 32, 'instances_tested': 1, 'conjecture_holds': 1}
RESULT: FALSIFIED counterexample="rank < monotone circuit size" first_failing_seed=11

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The conjecture has been falsified with a single instance, which is insufficient to confirm the conjecture’s validity. This is a clear case of ‘n too small’, as only one instance was tested. The metric does not scale trivially with n, but the small sample size makes it difficult to draw conclusions about the conjecture’s true nature.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge falsified | original: The test results show that for at least one instance, the minimal matroid rank of the affine geometric loci of a k-CLIQUE is less than the size of the | next: Further investigation is needed to determine if the conjecture holds for larger instances or under different conditions.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17234
2	propose	ollama_remote	glm4:latest	0	0	14530
3	preregistration	ollama_remote	glm4:latest	0	0	9898
4	novelty	ollama_remote	glm4:latest	0	0	8709
5	novelty	ollama_remote	glm4:latest	0	0	11890
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25684
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15961
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12796
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9772
10	critic	ollama_remote	glm4:latest	0	0	12884
11	judge	ollama_remote	glm4:latest	0	0	9702

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 149060 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in [research/audit/536c41f3714d.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice

3. The full LLM transcripts are in `research/audit/536c41f3714d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/536c41f3714d.tar.gz` (if generated)